

Escape Protocol: A Pursuit-Evasion Game

Cole Rutledge

Department of Computer Science

CSC 375 Term Project

April 22nd, 2026

Abstract - This paper presents *Escape Protocol*, a pursuit evasion game on a dynamically directed, weighted graph implemented in C++ with the use of the SFML library for visuals and the GUI. The system models a pursuit evasion scenario in which not only do graph directions change but graph weights can be either positive or negative. By procedurally generating a small solar system the game provides a randomized setting under which gravitational forces determine edge weights and how a player approaches the game as the one being pursued. The pursuer calculates the shortest path to the player via the Bellman Ford algorithm and uses that to track them. It provides an extension on the classical interpretation of the cops and robbers style game by increasing its complexity and the agency of the player. This report discusses motivation, architecture, testing strategy, the current state of the project and future directions it could take.

Keywords - SFML, cops and robbers, directed graph, weighted graph, Bellman Ford, pursuit-evasion

Code Availability - The source code can be found via the following link <https://github.com/dcrutled/EscapeProtocol.git>

I. Introduction

My term project was originally solely inspired by Star Trek. It's a franchise I really love and I knew I needed a project that would keep me engaged given the length and scope so I went with something I knew. The setting being in space also presented me with a lot of interesting challenges to address. The problem I wanted to address originally was graph traversal on a weighted directed graph with negative edges. Nothing that's ground breaking but novel nonetheless. However, as I progressed further into the game's development it became apparent that the game frankly was boring. The problem itself was somewhat interesting but by the end of milestone two I felt as though I could do something more original and interesting. Thus was born the pursuit/evasion game *Escape Protocol* inspired simultaneously by Dr. Yackley and Dr. McCleman. It tackles the much more interesting problem of a pursuit evasion game on a directed graph (whose directions are dynamic) with both positive and negative edge weights.

II. Related Works

The first of my related works is a paper by Nowakowski [1] that introduces what a cops and robbers style game is in terms of game theory and on a graph. It does not endeavour to really solve the problem especially on a weighted graph but introduces it. The next work I considered is far more interesting and directly relevant by Maddahai and Masehian [2]; it explores solving Cops and Robbers via Linguistic Geometry. And discusses what a cops and robbers game looks like in a multi agent setting, and how by breaking the overarching system into smaller subsystems an agent or multiple can solve the pursuit in cops and robbers. I would also be remiss not to mention a paper by Merz [3] that introduces the specific type of cops and robbers game mine most closely resembles which is the Homicidal Chauffeur. This is quite an old paper at this point but it introduces the homicidal chauffeur well enough. I also referenced the wolfram alpha page [4] on the homicidal chauffeur that features a simulation to help illustrate why the problem is interesting.

And for more interesting math on Cops and Robbers [5] delves into the actual game theoretical perspective that is important to have if one attempts to solve a pursuit evasion game.

Compared to these works, mine at this point is relatively scant but I believe it is differentiated by the inclusion of positive and negative edge weights with a dynamically directed graph. Because as the pursuer moves the graph edge directions change in the background which eventually will change the edge weights themselves but I've not gotten quite that far. So at this point my work is headed in a unique direction. shout out to Dr. McCleman for actually making me aware of the fact that this could be an interesting path to pursue, pun intended.

III. Approach

(I've included a table at the beginning of this section that helps to understand the function of each file that can be referenced during the following section. It should help to keep things straight for the reader rather than mentioning some specific file and expecting them to remember its function at some later point in the discussion.)

File Name	Purpose
main.cpp	Keeps track of game clock, window and game loop
GameSpace.h	Builds most game structures, graph, celestial bodies, constructs player and enemy and parses player movement among other things
GameState.h	Small file to determine which window to display.
player.h	Player entity
enemy.h	Enemy entity
Stack.h	Homemade stack ADT
Point.h	Node object for each node in graph.
Star.h	Star object that's shared amongst every window
StartScreen.h	Start screen GUI
LevelSelect.h	LEvel select GUI
GameOver.h	GameOver GUI
CelestialObject.h	Object for planets, blackholes, and other celestial objects.

The game is structured around one large GameSpace file, this determines the game map itself and procedurally generates any obstacles or boons therein. For the map structure I decided to put it on a graph and also draw that out for the player, aesthetically it looks complex enough to interest the player without becoming too overwhelming. I chose to build the graph from an adjacency list because at the beginning of this project that seemed like the easiest way to build a graph and I've found myself sorely regretting that decision. As I have discovered that referencing any undirected edge or sifting through directed ones is much easier done in an adjacency matrix at least given the way I have organized my game logic and rules.

The graph size is determined by a number of rings chosen by the player at launch of the game and ring number

corresponds directly to node quantity and graph complexity. Ring zero is the origin node and then the node magnitude in each ring is 2^n , where n starts at 2 and increases by one at every odd ring number. Ring 1 = 4 nodes, ring 2 = 4 nodes, then Ring 3 = 2^3 nodes and the pattern continues thereafter.

The radial separation of each node in a given ring is determined by the following formula $\frac{2\pi}{2^n}$ and every even ring is offset by that amount. So every odd ring will have a node at 0 degrees and the even ring that succeeds it will have its first node offset positively by the aforementioned amount.

Once all of this has been calculated in the background the game displays a nice game map for the player drawn using the SFML library, this is the library I used to do all of my graphics including the background, animations, and selection screens. It's also what I use to determine key and mouse input from the player.

After the map is built the program will calculate gravitational values on each edge. I used the regular old gravitational force formula but changed the constant to make the values more meaningful to the player. To calculate gravity for an edge whose points reside in different rings the game assumes a player is moving from point one of the edge to point two and then compares the distance from each point in an edge to every planet. If the distance from point two is greater than that from point one the gravity value is deemed positive because the player is moving away from the planet and negative if the inverse is true. Then it averages those values and shows that to the player. And in the case of an edge whose points are in the same ring it takes the average of both the moving away from and moving towards values. For the different ring case I'll eventually change the gravity because it's no longer true that the player is always moving from point one to point two, that was a holdover from when the player could only move outward towards the edge of the map since that was the goal.

The other most interesting piece of the program is the newly included pursuer dubbed "the enemy" in the program code which makes use of the bellman ford algorithm. Bellman ford is a dynamic programming algorithm that determines the shortest path to any given node by continuously relaxing each possible edge it can traverse. That essentially means it begins by only accepting edges closest to it then expands outward and updates its best path based on the addition of new edges. It has to iterate quite a few times to actually find its path but on a small scale like this that won't really matter. This is how the enemy finds our player.

They begin on an outer edge and determine a shortest path then when the player moves the enemy recalculates and is allowed to move two edges on that path,

```

classDiagram
    class Base {
        +int ID
        +string Name
        +float Voltage
        +float Current
        +float Power
        +int Frequency
        +float Amplitude
        +float Phase
        +int Temperature
        +float Humidity
        +float Pressure
        +int X
        +int Y
        +int Z
        +GetBaseInfo()
        +SetBaseInfo()
        +GetBaseStatus()
    }
    class Power {
        +int ID
        +string Name
        +float Voltage
        +float Current
        +float Power
        +GetVoltage()
        +GetCurrent()
        +GetPower()
        +SetPower()
    }
    class ControlUnit {
        +int ID
        +string Name
        +float Frequency
        +float Amplitude
        +float Phase
        +GetFrequency()
        +GetAmplitude()
        +GetPhase()
        +SetFrequency()
    }
    class Sensor {
        +int ID
        +string Name
        +float Temperature
        +float Humidity
        +float Pressure
        +GetTemperature()
        +GetHumidity()
        +GetPressure()
        +SetTemperature()
    }
    class Display {
        +int ID
        +string Name
        +int X
        +int Y
        +int Z
        +GetX()
        +GetY()
        +GetZ()
        +SetX()
    }
    Base "1" -- "1" Power
    Base "1" -- "1" ControlUnit
    Base "1" -- "1" Sensor
    Base "1" -- "1" Display
  
```

- [1] R. J. Nowakowski and P. Winkler, "Vertex-to-vertex pursuit in a graph," vol. 43, no. 2–3, pp. 235–239, Jan. 1983, doi: [https://doi.org/10.1016/0012-365x\(83\)90160-7](https://doi.org/10.1016/0012-365x(83)90160-7).
- [2] A. Maddahi and E. Masehian, "Linguistic Geometry Approach for Solving the Cops and Robber Problem in Grid Environments," *Information Sciences*, vol. 414, pp. 68–101, Nov. 2017, doi: <https://doi.org/10.1016/j.ins.2017.05.045>.
- [3] A. Merz, "The Homicidal Chauffeur - A Differential Game," Mar. 1971.

<https://scispace.com/pdf/the-homicidal-chauffeur-a-different-ial-game-3so6375d7f.pdf>

[4] “Wolfram Demonstrations Project,” *Wolfram.com*, 2026.
<https://demonstrations.wolfram.com/TheHomicidalChauffeurProblem/> (accessed Apr. 24, 2026).

[5] A. Frieze, M. Krivelevich, and P.-S. Loh, “Variations on Cops and Robbers,” *Journal of Graph Theory*, vol. 69, no. 4, pp. 383–402, Feb. 2011, doi:
<https://doi.org/10.1002/jgt.20591>.